Szymon Czop
[Akademia Marynarki Wojennej im. Bohaterów Westerplatte]

SYSTEM MONITOROWANIA SIECI IOT Z KLASYFIKACJĄ ATAKÓW METODAMI UCZENIA MASZYNOWEGO I MAPOWANIEM NA KATALOG TECHNIK I TAKTYK MITRE ATT&CK

STRESZCZENIE

Artykuł opisuje projekt IoT Security Radar — lokalny system monitorowania sieci IoT z detekcją i klasyfikacją ataków w czasie rzeczywistym. System łączy stos Elastic (Filebeat, Logstash, Elasticsearch, Kibana) z modułem uczenia maszynowego (Random Forest), osiągając 97% dokładność klasyfikacji dla 9 klas zdarzeń sieciowych. Zastosowana architektura dwuindeksowa oddziela surowe dane zdarzeń od predykcji modelu, umożliwiając niezależną analizę danych i decyzji klasyfikatora. Każde zdarzenie ataku jest mapowane na taktyki i techniki frameworka MITRE ATT&CK, co dostarcza analitykom centrum bezpieczeństwa (SOC) kontekstu niezbędnego do priorytetyzacji reagowania. W procesie walidacji przetworzono łącznie 55 280 zdarzeń pochodzących z trzech źródeł: zbioru NF-ToN-IoT-v2, symulowanych ataków oraz ruchu rzeczywistego przechwyconego przez tshark.

Słowa kluczowe:

bezpieczeństwo IoT, ML, MITRE ATT&CK, monitorowanie sieci

WSTĘP

Dynamiczny wzrost liczby urządzeń Internetu Rzeczy (IoT) — szacowany na ponad 18 miliardów aktywnych urządzeń w 2025 roku[1] — tworzy rozległą powierzchnię ataku, szczególnie narażoną na działania cyberprzestępców. Urządzenia IoT korzystają z różnorodnych protokołów komunikacyjnych, co znacząco utrudnia ich ochronę przy użyciu klasycznych narzędzi bezpieczeństwa sieci. Istniejące rozwiązania klasy SIEM (Security Information and Event Management), choć oferują szerokie możliwości korelacji zdarzeń, są zazwyczaj przeznaczone dla dużych infrastruktur korporacyjnych — koszty ich licencji i złożoność wdrożeniowa wykluczają je z zastosowań w małych sieciach domowych i laboratoryjnych. Brakuje rozwiązania, które w jednym środowisku łączyłoby zbieranie ruchu, klasyfikację ML i kontekst operacyjny dostarczany przez mapowanie na katalog taktyk i technik MITRE ATT&CK.

Celem niniejszego artykułu jest zaprojektowanie, wdrożenie i walidacja lokalnego systemu monitorowania sieci IoT, który automatycznie klasyfikuje ataki za pomocą metod uczenia maszynowego i mapuje zdarzenia na taktyki oraz techniki MITRE ATT&CK.

---

[1] Bitdefender i NETGEAR, *2025 IoT Security Landscape Report*, Bitdefender, 2025, https://www.bitdefender.com/en-us/blog/hotforsecurity/bitdefender-and-netgear-2025-iot-security-landscape-report-shows-alarming-rise-in-smart-home-threats, dostęp 01.03.2026 r.

Przedmiotem rozważań jest bezpieczeństwo sieci IoT, ze szczególnym uwzględnieniem możliwości detekcji i klasyfikacji zagrożeń w ruchu sieciowym bez użycia zewnętrznych usług chmurowych.

Wkładem własnym autora jest projekt i implementacja kompletnego systemu łączącego transformacje ETL (Extract, Transform, Load) oparte na Elastic Stack z klasyfikatorem ML oraz operacyjnym mapowaniem na katalog taktyk i technik MITRE ATT&CK, a także jego walidacja na zbiorze referencyjnym NF-ToN-IoT-v2 i rzeczywistym ruchu sieciowym przechwyconym przez tshark. Pełny kod źródłowy systemu jest publicznie dostępny w repozytorium GitHub.[16]

W pracy zastosowano metodologię eksperymentalną: zbudowano działający system, przetestowano go na zbiorze danych referencyjnych (NF-ToN-IoT-v2) oraz na rzeczywistym ruchu sieciowym, a następnie porównano wyniki klasyfikatora z prawdziwymi etykietami. Analizę porównawczą uzupełniono wnioskowaniem przez analogię — spostrzeżenia ze środowiska laboratoryjnego odniesiono do scenariuszy produkcyjnych. Artykuł jest podzielony na pięć rozdziałów: charakterystykę zagrożeń IoT, opis architektury systemu, zbieranie i przetwarzanie danych, klasyfikację ML oraz weryfikację systemu w warunkach zbliżonych do rzeczywistych.

1. ZAGROŻENIA BEZPIECZEŃSTWA W SIECIACH IOT

1.1 Specyfika zagrożeń IoT

Urządzenia IoT funkcjonują w warunkach fundamentalnie odmiennych od klasycznych stacji roboczych czy serwerów. Ograniczone zasoby obliczeniowe, brak mechanizmów aktualizacji oprogramowania oraz niejednorodność protokołów komunikacyjnych sprawiają, że tradycyjne narzędzia bezpieczeństwa sieci — takie jak agenty endpoint detection and response (EDR) — są niepraktyczne lub niemożliwe do wdrożenia na samych urządzeniach. Ochrona musi odbywać się na poziomie sieci.

Na podstawie analizy zbioru NF-ToN-IoT-v2[2] oraz przeglądu literatury wyróżniono następujące kategorie ataków charakterystyczne dla sieci IoT:

– ataki DDoS/DoS — przeciążenie zasobów sieciowych i obliczeniowych urządzeń;

– brute force (SSH, HTTP) — łamanie haseł przez iterację słownikową;

– skanowanie portów (port scan) — rozpoznanie topologii sieci i otwartych usług;

– ataki injection (SQL, command injection) — wstrzykiwanie złośliwego kodu przez podatne interfejsy;

– DNS exfiltration — wyprowadzanie danych kanałem DNS;

– XSS (Cross-Site Scripting) — ataki na interfejsy webowe urządzeń zarządzających;

– backdoor — instalacja trwałego dostępu poprzez webshell lub zmodyfikowane oprogramowanie układowe.

---

[2]M. Sarhan, S. Layeghy, M. Portmann, *Towards a Standard Feature Set for Network Intrusion Detection System Datasets*, „Mobile Networks and Applications”, 2022, vol. 27, s. 357–370.

Badania Koroniotis i wsp. wskazują, że ataki na sieci IoT są coraz częściej prowadzone w sposób automatyczny przez botnety złożone z zainfekowanych urządzeń, co dodatkowo utrudnia ich wykrywanie na podstawie statycznych wzorców.[3]

Tabela 1 zestawia najpopularniejsze rozwiązania monitorowania bezpieczeństwa dostępne bez kosztów licencyjnych w zastosowaniach domowych i laboratoryjnych pod kątem kryteriów istotnych dla niniejszego projektu.

Tabela 1. Porównanie wybranych narzędzi monitorowania bezpieczeństwa sieci w zastosowaniach domowych i laboratoryjnych

Narzędzie	Koszt (home/lab)	Wymaga chmury	ML / klasyfikacja własna	Mapowanie ATT&CK	Min. RAM
Splunk Free	Bezpłatny; limit 500 MB/dzień	Nie	Nie	Nie	8 GB
Wazuh	Bezpłatny (open-source)	Opcjonalnie	Nie	Reguły statyczne	8 GB
Security Onion 2.4	Bezpłatny (open-source)	Nie	Nie	Navigator (wizualizacja)	16 GB
Elastic Stack (self-hosted)	Bezpłatny; ML wymaga Platinum (131 USD/mies.)	Nie	Anomaly detection (Platinum+)	Reguły wbudowane	8 GB
IoT Security Radar (niniejszy system)	**Bezpłatny (open-source)**	**Nie**	**Random Forest, 97% acc.**	**Dynamiczne (per zdarzenie)**	**8 GB**

Żadne z zestawionych rozwiązań nie oferuje jednocześnie własnego klasyfikatora ML przepisującego wieloklasowe etykiety ataków oraz dynamicznego mapowania wyników na taktyki MITRE ATT&CK bez dodatkowych kosztów licencyjnych. Splunk Free wyczerpywałby dzienny limit przy zbiorze 55 280 zdarzeń użytym w niniejszej pracy w czasie poniżej godziny.11 Security Onion wymaga minimum 16 GB RAM, co wyklucza zastosowanie na typowym sprzęcie domowym.12 Luka ta stanowi bezpośrednie uzasadnienie dla budowy dedykowanego systemu opisanego w niniejszej pracy.

1.2 Framework MITRE ATT&CK jako narzędzie klasyfikacji

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) jest publicznie dostępnym systemem klasyfikacji zagrożeń opisującym zachowania atakujących na każdym etapie operacji ofensywnych.[4] Systematyzuje on wiedzę o taktykach, technikach oraz procedurach. Zastosowanie systemu ATT&CK do kategoryzacji

---

[3]N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, *Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics*, „Future Generation Computer Systems", 2019, vol. 100, s. 779–796.

[4]MITRE Corporation, *MITRE ATT&CK: Design and Philosophy*, MITRE Corporation, 2020, https://attack.mitre.org/, dostęp 01.03.2026 r.

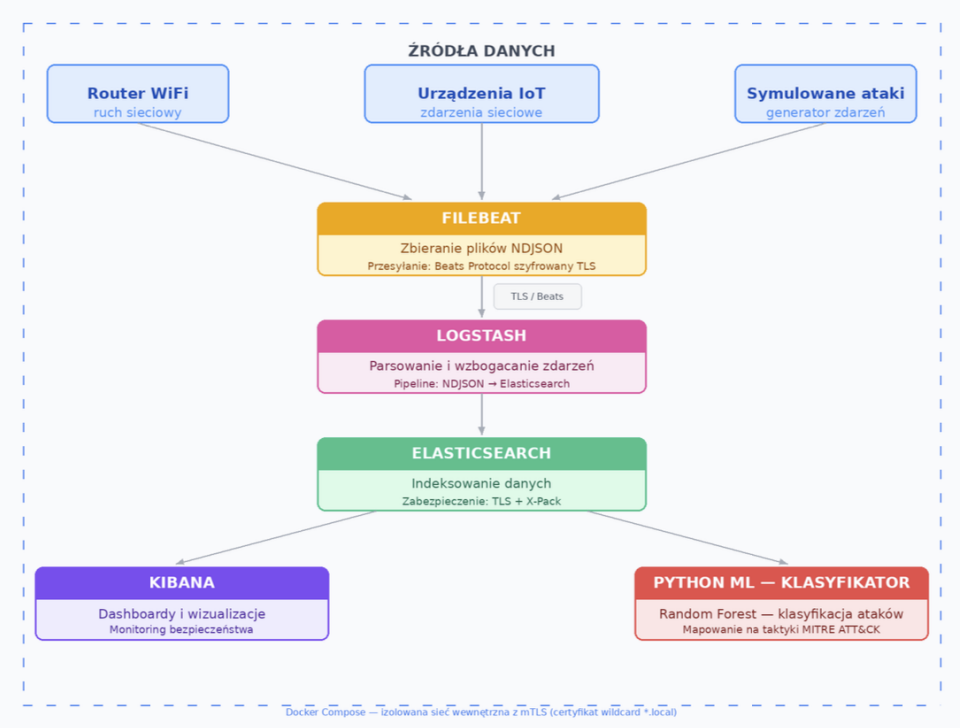
incydentów dla wyników klasyfikatora ML pozwala przekształcić surowy alert sieciowy w informację operacyjną zrozumiałą dla analityka SOC.

W niniejszym systemie każda klasa ataku jest mapowana na odpowiednią taktykę i technikę ATT&CK. W tabeli 3 (rozdział 4) zestawiono kompletne mapowanie dla dziewięciu klas klasyfikatora wraz z poziomem krytyczności (severity), który jest podstawą priorytetyzacji alertów. Użyte techniki obejmują m.in. T1046 (Discovery), T1110.001 (Credential Access), T1048.001 (Exfiltration), T1498.001 i T1499.001 (Impact), T1059 (Execution) oraz T1505.003 (Persistence).

2. ARCHITEKTURA SYSTEMU IOT SECURITY RADAR

2.1 Przegląd komponentów

System IoT Security Radar składa się z czterech warstw logicznych: zbierania danych, przetwarzania i indeksowania, klasyfikacji ML oraz wizualizacji i alarmowania krytycznych incydentów. Przepływ danych ilustruje rysunek 1.



Rys. 1. Architektura systemu IoT Security Radar (opracowanie własne)

2.2 Elastic Stack i bezpieczeństwo komunikacji

Elasticsearch pełni rolę centralnego repozytorium zdarzeń.[5] Skonfigurowany w trybie single-node z włączonymi mechanizmami xpack.security, indeksuje zdarzenia w formacie NDJSON. Dwa indeksy logiczne — iot-radar-* (surowe zdarzenia z etykietami ground-truth) oraz iot-radar-predictions (wyniki klasyfikatora ML) — realizują architekturę dwuindeksową opisaną szczegółowo w rozdziale 3.

---

[5]Elastic N.V., *Elastic Stack Documentation 8.x*, Elastic N.V., 2024, https://www.elastic.co/guide/, dostęp 01.03.2026 r.

Filebeat zbiera pliki NDJSON z katalogu sample-logs/ i przesyła je do Logstasha przez szyfrowany kanał TLS (certyfikat wildcard *.local).

Logstash przetwarza zdarzenia w pipeline: wejście beats (port 5044, mTLS) → filtr (normalizacja pól, tagowanie źródła data_source) → wyjście Elasticsearch.

Kibana udostępnia dwa dashboardy: SOC Dashboard (dane referencyjne, rozkład typów ataków, mapa taktyk MITRE) oraz ML Predictions Dashboard (predykcje modelu, poziom pewności klasyfikatora, rozkład po taktykach ATT&CK). Alerty są wyzwalane automatycznie dla zdarzeń o wysokim i krytycznym poziomie zagrożenia.

Bezpieczeństwo komunikacji zapewniają wzajemne uwierzytelnianie TLS (mTLS) z certyfikatami PEM oraz mechanizm xpack.security Elasticsearch z dedykowanymi kontami (elastic, kibana). Wszystkie usługi są izolowane w sieci Docker,[6] bez ekspozycji portów wewnętrznych na zewnątrz.

2.3 Środowisko testowe

Wszystkie komponenty systemu uruchomiono lokalnie na komputerze Apple z procesorem ARM64 (Apple Silicon, 10 rdzeni fizycznych), wyposażonym w 16 GB zunifikowanej pamięci RAM, z systemem operacyjnym macOS 26.0.1 (build 25A362). Środowisko kontenerowe oparto na Docker Desktop 29.2.0 z Docker Compose v5.0.2; silnik Docker udostępniał kontenerom łącznie 7,65 GB RAM.

Łączna zajętość dysku przez obrazy Docker, indeksy Elasticsearch i pliki konfiguracyjne wynosiła 2,9 GB. Podczas pełnego cyklu demonstracyjnego (55 280 zindeksowanych zdarzeń) zmierzone zużycie pamięci przez poszczególne kontenery przedstawiało się następująco: Elasticsearch — 1,87 GB, Logstash — 1,05 GB, Kibana — 643 MB, Filebeat — 53 MB. Łączne zużycie RAM przez stos Elastic nie przekraczało 3,6 GB, co stanowi 47% zasobów udostępnionych przez Docker i potwierdza możliwość uruchomienia systemu w typowych warunkach domowych i laboratoryjnych bez dedykowanego serwera fizycznego.13

3. ZBIERANIE I PRZETWARZANIE DANYCH SIECIOWYCH

3.1 Metody pozyskiwania danych

W systemie zaimplementowano trzy ścieżki pozyskiwania danych o różnym charakterze i przeznaczeniu.

Przechwytywanie ruchu rzeczywistego realizuje skrypt oparty na narzędziu tshark.[7] Skrypt korzysta z podejścia flow-based: pakiety są grupowane w przepływy (src_ip, dst_ip, src_port, dst_port, protocol), a cechy takie jak liczba bajtów przychodzących i wychodzących są sumowane dla całego przepływu. Podejście flow-based, w odróżnieniu od analizy pojedynczych pakietów, umożliwia poprawne obliczenie zagregowanych statystyk ruchu niezbędnych dla klasyfikatora.

---

[6]Docker Inc., *Docker Documentation*, Docker Inc., 2024, https://docs.docker.com/, dostęp 01.03.2026 r.

[7]Wireshark Foundation, *TShark — Terminal-based Wireshark*, Wireshark Foundation, 2024, https://www.wireshark.org/docs/man-pages/tshark.html, dostęp 01.03.2026 r.

Zbiór referencyjny NF-ToN-IoT-v2 (Koroniotis i wsp., 2019) zawiera 16,9 mln przepływów sieciowych przechwyconych w laboratorium cyber-range Uniwersytetu w Canberze.[8] Zbiór obejmuje 14 klas ataków i ruch normalny; do treningu i walidacji modelu użyto zbalansowanego podzbioru 55 000 próbek. Istotna cecha zbioru — adresy IP źródłowe w zakresie 192.168.1.x zarówno dla ruchu normalnego, jak i atakującego — wynika z topologii laboratorium, w której atakujący i ofiary współdzielą podsieć.

Symulowane ataki generuje dedykowany skrypt, który produkuje zdarzenia NDJSON z wbudowanymi etykietami. Każde zdarzenie jest opatrzone znacznikiem źródła, co umożliwia odróżnienie danych symulowanych od zbioru referencyjnego i ruchu rzeczywistego na poziomie filtrów zaimplementowanych w Kibana.

3.2 Pipeline przetwarzania

Architektura dwuindeksowa jest kluczowym elementem projektowym systemu. Indeks iot-radar-* przechowuje surowe zdarzenia z rzeczywistymi etykietami nadanymi przez skrypty generujące dane. Indeks iot-radar-predictions zawiera wyłącznie wyniki działania klasyfikatora ML; w tym pola opisujące predykcję klasyfikatora, poziom pewności, przypisaną taktykę i technikę MITRE ATT&CK oraz poziom zagrożenia.

Rozdzielenie obu indeksów umożliwia niezależną ocenę skuteczności modelu: metryki (accuracy, F1) są obliczane wyłącznie dla danych wzorcowych z etykietami referencyjnymi, natomiast zdarzenia tshark-live — pozbawione etykiet referencyjnych— są klasyfikowane przez model, lecz nie wchodzą do zbioru ewaluacyjnego. Architektura ta eliminuje ryzyko przepływu danych między etapem ewaluacji a etapem inferencji, co jest szczególnie istotne przy iteracyjnym doszkalaniu modelu na danych produkcyjnych.

4. KLASYFIKACJA ATAKÓW Z WYKORZYSTANIEM UCZENIA MASZYNOWEGO

4.1 Wybór algorytmu i przygotowanie danych

Do klasyfikacji ataków wybrano algorytm Random Forest (Breiman, 2001),[9] kierując się trzema kryteriami: interpretowalność (możliwość oceny wkładu poszczególnych cech w decyzję klasyfikatora), odporność na szum i brakujące wartości oraz brak konieczności standaryzacji danych numerycznych. Rozwiązania oparte na sieciach neuronowych oferują potencjalnie wyższą dokładność, lecz wymagają znacznie większych zasobów obliczeniowych i są trudniejsze do zinterpretowania w kontekście operacyjnym SOC.

Cechy wejściowe klasyfikatora odpowiadają polom przepływu sieciowego NetFlow: src_ip (numerycznie), dst_ip, src_port, dst_port, protocol, bytes_in, bytes_out, duration, packets_in, packets_out oraz flaga src_toniot_benchmark. Zestaw cech

---

[8] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, *Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics*, „Future Generation Computer Systems", 2019, vol. 100, s. 779–796.

[9] L. Breiman, *Random Forests*, „Machine Learning", 2001, vol. 45, s. 5–32.

uzupełniono o binarną flagę wskazującą przynależność zdarzenia do zbioru referencyjnego, która w środowisku produkcyjnym powinna zostać usunięta.

Mapowanie etykiet klas na liczby całkowite realizowane jest z użyciem biblioteki scikit-learn (Pedregosa i wsp., 2011),[10] z zachowaniem stałego porządku alfabetycznego klas. Kodowanie etykiet jest zapisywane razem z modelem, co zapewnia spójność mapowania między treningiem a inferencją — niezbędny warunek poprawnego działania klasyfikatora na nowych danych.

Wybór algorytmu Random Forest zamiast bardziej złożonych modeli, takich jak gradient boosting (XGBoost, LightGBM) czy głębokie sieci neuronowe, jest celowy i uzasadniony charakterem zadania. Wymienione algorytmy oferują potencjalnie wyższą dokładność kosztem znacznie dłuższego czasu treningu, większego zużycia pamięci operacyjnej oraz trudniejszej interpretacji wyników — cech szczególnie istotnych w środowisku kontenerowym z ograniczonymi zasobami. Uzyskana dokładność ogólna na poziomie 97% przy F1=1.00 dla czterech z dziewięciu klas wskazuje, że wzorce tych ataków są jednoznacznie rozróżnialne w danych treningowych, a złożoność modelu nie jest czynnikiem ograniczającym skuteczność systemu. W takich warunkach zwiększanie złożoności klasyfikatora przynosi marginalne korzyści przy wymiernym wzroście kosztów obliczeniowych i utracie interpretowalności — właściwości krytycznej dla operatora SOC, który musi rozumieć podstawę każdego alertu.

4.2 Wyniki klasyfikacji

Model osiągnął dokładność ogólną 97% na zbiorze testowym (11 000 próbek). Szczegółowe wyniki dla poszczególnych klas prezentuje tabela 2.

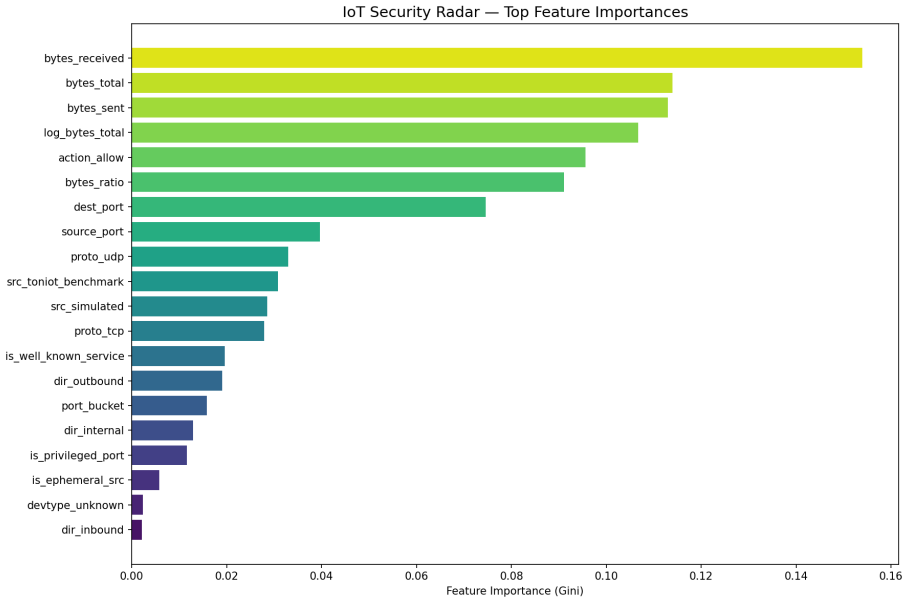
Tabela 2. Raport klasyfikacji — Random Forest (zbiór testowy, n=11 000)

Klasa	Precision	Recall	F1-score	Support
backdoor	1.00	1.00	1.00	7
brute_force	0.90	0.88	0.89	580
ddos_flood	0.96	0.93	0.94	956
dns_exfiltration	1.00	1.00	1.00	9
dos	0.89	0.91	0.90	338
injection	0.74	0.76	0.75	322
normal	1.00	1.00	1.00	5900
port_scan	1.00	0.99	1.00	1777
xss	0.92	0.95	0.93	1111

[10] F. Pedregosa i in., *Scikit-learn: Machine Learning in Python*, „Journal of Machine Learning Research”, 2011, vol. 12, s. 2825–2830.

Uzyskana dokładność 97% jest zbieżna z wynikami raportowanymi w literaturze dla zbiorów NetFlow na podobnych danych — Sarhan i wsp. osiągają porównywalne metryki przy użyciu zunifikowanego zestawu cech NetFlow na zbiorze NF-ToN-IoT-v2.[11] Przewaga zastosowanego podejścia leży nie w samej dokładności klasyfikacji, lecz w integracji potoku zbierania danych, klasyfikatora i mapowania operacyjnego ATT&CK w jednym spójnym systemie gotowym do wdrożenia.

Klasy injection (F1=0.75) i brute_force (F1=0.89) osiągają niższe wyniki z powodu nakładania się rozkładów cech z innymi klasami ataków aplikacyjnych. Klasy normal, backdoor, dns_exfiltration i port_scan uzyskują F1=1.00, co oznacza, że klasyfikator rozpoznaje je bezbłędnie. Rysunek 2 przedstawia ważność cech (mean decrease in impurity) dla 15 najistotniejszych zmiennych wejściowych.



Rys. 2. Ważność cech klasyfikatora Random Forest (top 15) (opracowanie własne)

4.3 Mapowanie predykcji na MITRE ATT&CK

Każda klasa ataku jest statycznie mapowana na taktykę, technikę i poziom zagrożenia zgodnie z katalogiem taktyk i technik MITRE ATT&CK przy użyciu dedykowanego pliku konfiguracyjnego. Mapowanie prezentuje tabela 3.

Tabela 3. Mapowanie klas ataków na MITRE ATT&CK

Klasa ataku	Taktyka	Technika	Nazwa techniki	Severity

[11] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, *Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics*, „Future Generation Computer Systems", 2019, vol. 100, s. 779–796.

backdoor	Persistence	T1505.003	Server Software Component: Web Shell	critical
brute_force	Credential Access	T1110.001	Brute Force: Password Guessing	high
ddos_flood	Impact	T1498.001	Network Denial of Service: Direct Flood	critical
dns_exfiltration	Exfiltration	T1048.001	Exfiltration Over Alternative Protocol	critical
dos	Impact	T1499.001	Endpoint Denial of Service: OS Exhaustion	high
injection	Execution	T1059	Command and Scripting Interpreter	critical
normal	---	---	---	---
port_scan	Discovery	T1046	Network Service Discovery	medium
xss	Initial Access	T1189	Drive-by Compromise	high

W dashboardzie Kibana alerty są wyzwalane automatycznie dla zdarzeń o poziomie krytycznym (backdoor, ddos_flood, dns_exfiltration, injection) oraz wysokim (brute_force, dos, xss). Mapowanie jest przechowywane w dedykowanym pliku konfiguracyjnym, co ułatwia jego rozszerzenie o nowe klasy ataków bez modyfikacji kodu klasyfikatora.

5. WERYFIKACJA I DEMONSTRACJA SYSTEMU

5.1 Demonstracja w czasie rzeczywistym

Kompletny cykl demonstracyjny realizuje dedykowany skrypt demonstracyjny, uruchamiany z parametrami określającymi czas trwania sesji (domyślnie 3 minuty) oraz liczbę generowanych zdarzeń ataku (domyślnie 300).

Skrypt równolegle przechwytuje ruch WiFi przez tshark oraz generuje symulowane zdarzenia ataku, zapisując wyniki do pliku NDJSON. Filebeat wykrywa zmianę pliku i przesyła nowe zdarzenia przez mTLS do Logstasha, który je indeksuje do Elasticsearch. Moduł ML pobiera świeże zdarzenia przez klienta Elasticsearch, klasyfikuje je i zapisuje predykcje do indeksu iot-radar-predictions. Cały cykl od przechwycenia pakietu do pojawienia się predykcji w Kibana zamyka się w czasie poniżej 2 minut.

W toku weryfikacji zarejestrowano łącznie 55 280 zdarzeń ze źródeł:

– toniot_benchmark: 50 000 zdarzeń (NF-ToN-IoT-v2, balans 50/50 normal/attack);

– simulated: 5 000 zdarzeń (generuje dedykowany skrypt, stosunek 90/10);

– tshark-live: 265 zdarzeń (rzeczywisty ruch WiFi, brak ground-truth ataków).

5.2 Dashboardy Kibana

Uzyskane wyniki można również zwizualizować w formie graficznej. Poniżej przedstawiono możliwości zaimplementowane w pierwszej fazie projektu z wykorzystaniem platformy Kibana.

Jeden z wykresów w Kibanie (SOC Dashboard), oparty na indeksie iot-radar-* prezentuje rozkład typów ataków, oś czasu zdarzeń, mapę taktyk MITRE oraz tabelę źródłowych IP najczęściej wykorzystanych podczas ataku. Dashboard ten opiera się na etykietach referencyjnych, umożliwiając analizę sklasyfikowanych danych.

ML Predictions Dashboard oparty na indeksie iot-radar-predictions, prezentuje wyniki działania modelu: rozkład prognozowanych klas, rozkład pewności klasyfikatora, rozkład zdarzeń według katalogu taktyk i technik MITRE ATT&CK oraz historię alertów. Dla zdarzeń pochodzących z ruchu rzeczywistego jest to jedyne miejsce w systemie, w którym uzyskuje on klasyfikację bezpieczeństwa — dane nieprzetworzone nie zawierają etykiet referencyjnych.

Walidacja systemu obejmuje dwie perspektywy: ilościową — osiągnięto 97% celności na zbiorze referencyjnym z etykietami referencyjnymi, potwierdzonym szczegółowym raportem klasyfikacyjnym — oraz jakościową, przeprowadzoną na około 265 rzeczywistych przepływach sieci WiFi pozbawionych etykiet. System zwalidowano na 55 000 etykietowanych zdarzeniach, a następnie zastosowano wobec około 265 nie etykietowanych przepływów, dla których dane referencyjne nie były dostępne.

WNIOSKI

System IoT Security Radar skutecznie realizuje zamierzony cel: lokalne monitorowanie sieci IoT z detekcją i klasyfikacją ataków w czasie rzeczywistym, bez zależności od zewnętrznych usług chmurowych. Klasyfikator Random Forest osiąga dokładność 97% na zbiorze 11 000 próbek testowych, a architektura dwuindeksowa umożliwia niezależną ewaluację modelu oraz analizę operacyjną w odniesieniu do katalogu taktyk i technik MITRE ATT&CK.

Spośród zidentyfikowanych ograniczeń systemu na uwagę zasługuje statyczne mapowanie katalogu taktyk i technik MITRE ATT&CK, które nie uwzględnia dynamicznych łańcuchów ataków wieloetapowych.

Istotną cechą architektury systemu jest jego pełna autonomia infrastrukturalna: wszystkie komponenty działają lokalnie w izolowanej sieci Docker bez zależności od zewnętrznych dostawców usług chmurowych. Podejście to ma dwie praktyczne konsekwencje. Po pierwsze, administrator systemu zachowuje pełną kontrolę nad kierunkiem rozwoju — nowe źródła danych, dodatkowe klasy ataków, alternatywne algorytmy klasyfikacji czy integracja z systemami alarmowymi mogą być wdrażane bez ograniczeń licencyjnych i bez negocjacji z dostawcą. Po drugie, dane sieciowe — ze swej natury wrażliwe, gdyż odzwierciedlają topologię i aktywność chronionej infrastruktury — pozostają wyłącznie pod kontrolą operatora. Przechowywanie tego rodzaju danych u zewnętrznych dostawców SaaS niesie realne ryzyko ich ujawnienia: w 2024 roku naruszenia danych w środowiskach chmurowych stanowiły 45% wszystkich potwierdzonych incydentów bezpieczeństwa,[14] a ataki na platformy SaaS za pośrednictwem skradzionych poświadczeń umożliwiały jednoczesny dostęp do danych setek klientów danego dostawcy.[15]

Rekomendacje dla dalszego rozwoju systemu obejmują: zastąpienie przechwytywania pakietów protokołem NetFlow/IPFIX jako docelowym źródłem cech

przepływowych; rozszerzenie zbioru treningowego o dane produkcyjne w celu ograniczenia wycieku informacji między zbiorami; implementację modelu wieloklasowego z możliwością detekcji łańcuchów ataków wieloetapowych; oraz integrację z komercyjnym systemem SIEM poprzez interfejs programistyczny. System w obecnej postaci stanowi funkcjonalne i w pełni odtwarzalne środowisko badawcze, odpowiednie do dalszych eksperymentów w obszarze bezpieczeństwa IoT.

BIBLIOGRAFIA

1. Breiman L., Random Forests, „Machine Learning", 2001, vol. 45, s. 5–32.
2. Docker Inc., Docker Documentation, Docker Inc., 2024, https://docs.docker.com/, dostęp 01.03.2026 r.
3. Elastic N.V., Elastic Stack Documentation 8.x, Elastic N.V., 2024, https://www.elastic.co/guide/, dostęp 01.03.2026 r.
4. Koroniotis N., Moustafa N., Sitnikova E., Turnbull B., Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics, „Future Generation Computer Systems", 2019, vol. 100, s. 779–796.
5. MITRE Corporation, MITRE ATT&CK: Design and Philosophy, MITRE Corporation, 2020, https://attack.mitre.org/, dostęp 01.03.2026 r.
6. Moustafa N., Slay J., UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems, Military Communications and Information Systems Conference (MilCIS), IEEE, Canberra 2015, s. 1–6.
7. Pedregosa F. i in., Scikit-learn: Machine Learning in Python, „Journal of Machine Learning Research", 2011, vol. 12, s. 2825–2830.
8. Sarhan M., Layeghy S., Portmann M., Towards a Standard Feature Set for Network Intrusion Detection System Datasets, „Mobile Networks and Applications", 2022, vol. 27, s. 357–370.
9. Bitdefender i NETGEAR, *2025 IoT Security Landscape Report*, Bitdefender, 2025, https://www.bitdefender.com/en-us/blog/hotforsecurity/bitdefender-and-netgear-2025-iot-security-landscape-report-shows-alarming-rise-in-smart-home-threats, dostęp 01.03.2026 r.
10. Wireshark Foundation, TShark — Terminal-based Wireshark, Wireshark Foundation, 2024, https://www.wireshark.org/docs/man-pages/tshark.html, dostęp 01.03.2026 r.
11. CloudWize, *Cloud Data Breaches in 2024: A Year in Review*, CloudWize, 2025, https://www.cloudwize.io/blog/cloud-data-breaches-in-2024-a-year-in-review/, dostęp 15.03.2026 r.
12. Reco.ai, *AI & Cloud Security Breaches: 2025 Year in Review*, Reco.ai, 2025, https://www.reco.ai/blog/ai-and-cloud-security-breaches-2025, dostęp 15.03.2026 r.

13. Security Onion Solutions, *Security Onion Documentation 2.4*, Security Onion Solutions, 2025, https://docs.securityonion.net/en/2.4/, dostęp 15.03.2026 r.

14. Splunk Inc., *Types of Splunk Enterprise Licenses*, Splunk Inc., 2025, https://docs.splunk.com/Documentation/Splunk/9.4.0/Admin/TypesofSplunklicenses, dostęp 15.03.2026 r.

15. Wazuh Inc., *Wazuh — Open Source XDR*, Wazuh Inc., 2025, https://wazuh.com/, dostęp 15.03.2026 r.

16. Czop S, *IoT Security Radar,* GitHub, 2026, https://github.com/szymonczop/iot-security-radar

IOT NETWORK MONITORING SYSTEM WITH ML-BASED ATTACK CLASSIFICATION AND MITRE ATT&CK FRAMEWORK MAPPING

ABSTRACT

This paper presents IoT Security Radar — a local IoT network monitoring system with real-time attack detection and classification. The system integrates the Elastic Stack (Filebeat, Logstash, Elasticsearch, Kibana) with a machine learning module based on the Random Forest algorithm, achieving 97% classification accuracy across nine attack classes. A dual-index architecture separates raw events (with ground-truth labels) from ML predictions, enabling independent evaluation of model performance and ground-truth data quality. Each detected attack is mapped to MITRE ATT&CK tactics and techniques, providing SOC operators with operational context for alert prioritization. The system was validated on 55,280 indexed events from three sources: the NF-ToN-IoT-v2 benchmark dataset, simulated attacks, and live WiFi traffic captured via tshark. The entire stack is containerized with Docker Compose and secured with mutual TLS authentication.